ReduLink: Authenticated Reference Substitution for Redundancy-Suppressed Transfers over Encrypted QUIC Streams

Michél Nguyen
University of the People | minh.systems | ORCID: 0000-0001-6834-4422

Abstract

Encrypted WAN traffic prevents transparent redundancy elimination by middleboxes, but cooperating endpoints can still suppress repeated payload bytes before transport protection. ReduLink is an endpoint-controlled representation layer that replaces repeated chunks with authenticated references bound to epoch, scope, stream identity, offset, nonce, length, and chunk hash. The current artifact implements a compact binary ReduLink mapping inside TLS-protected aioquic streams, deterministic semantic MISS/FULL repair, authenticated frame validation, workload runners, and local component measurements. It deliberately does not claim custom QUIC extension frames or full network-emulator congestion fairness.

Across deterministic journal fixtures, ReduLink improves stream or modeled wire bytes only when byte-identical chunks survive across warm receiver state. It is strong for page-like snapshots and selected aligned update shapes, weak for shifted metadata and structured logs, and neutral or negative on independently curated public source-release pairs. The evidence therefore supports a conditional systems claim: authenticated reference substitution is deployable over encrypted endpoint transport and can increase effective reconstructed throughput for warm-state workloads, but it is not a universal replacement for compression, delta transfer, rsync, or congestion-control validation.

1. Introduction

Network capacity is normally described by physical or negotiated line rate. A software protocol cannot make a 1 Gbit/s Ethernet interface emit more signal than its physical layer permits, and modern Ethernet already targets far higher rates through IEEE 802.3 families [14]. What a software protocol can change is the relationship between bytes placed on the network and bytes reconstructed by the receiver. If endpoints share dictionary state, the sender can transmit references to earlier byte ranges instead of resending the full payload.

This idea is not new. Redundancy elimination, SmartRE, EndRE, LBFS, and rsync-family delta transfer all exploit repeated byte identity in different deployment positions [1-5]. The pressure point has changed. QUIC encrypts transport payloads, integrates loss recovery and congestion control, and intentionally reduces middlebox visibility [6-10]. Transparent WAN optimizers that rewrite encrypted transport payloads are no longer a natural deployment path. Any modern redundancy-suppression design must make the endpoint the authority for dictionary state, authentication, fallback, and privacy.

ReduLink studies that endpoint path. The design is intentionally narrower than a production QUIC extension: the current implementation maps ReduLink messages onto ordinary encrypted QUIC streams using aioquic [18]. This still exercises a real QUIC handshake, stream delivery, encrypted UDP packetization, loss handling, and application repair, while avoiding premature claims about custom frame parsing, transport-parameter negotiation, 0-RTT policy, or migration behavior.

The paper is also intentionally cautious about performance language. The metric of interest is effective reconstructed throughput: reconstructed application bytes divided by encoded stream or modeled wire bytes. It is not physical line-rate multiplication. All gains must be interpreted with the workload, dictionary scope, chunking policy, fallback overhead, and measurement layer in view.

2. Contributions and Claims

- An endpoint-controlled ReduLink abstraction for authenticated reference substitution in encrypted WAN settings.
- A security model binding references to epoch, scope, stream id, reconstructed offset, length, nonce, chunk identity, and payload hash.
- A compact binary ReduLink stream mapping over native aioquic QUIC streams, with semantic MISS and FULL repair.

- A workload-sensitive evaluation including deterministic journal fixtures, pinned public text fixtures, new independently curated public source-release pairs, and negative controls.
- A raw QUIC versus ReduLink-over-QUIC flow comparison under no-loss and deterministic-loss settings.
- Local component-cost measurements for fixed chunking, content-defined chunking, HMAC validation, and binary encode/decode.
- A reproducible package with tests, CI workflow, figures, checksums, reference audit, and artifact inventory [15].
- An exporter-style HKDF key schedule that binds ReduLink artifact keys to ALPN, epoch, scope, connection context, and stream context.
- A real-workload manifest runner for reviewer-supplied OCI layers, package updates, Git snapshots, VM images, or structured logs.
- A concurrent raw-QUIC versus ReduLink-over-QUIC smoke experiment that checks overlapping encrypted transfers and reconstruction under the artifact path.

The central claim is conditional: ReduLink improves effective reconstructed throughput for warm-state workloads with byte-stable repeated chunks under endpoint-controlled encrypted transport. The paper does not claim universal acceleration, better delta transfer than rsync, superiority over compression, or production-ready Deduplex-QUIC extension frames. The negative results are part of the claim boundary, not an appendix to be hidden.

3. Related Work and Gap

Classical redundancy elimination showed that network traffic contains repeated byte strings at useful scales and that suppressing those repeats can save bandwidth [1-3]. Those systems often assumed middlebox visibility or enterprise-controlled paths. They remain important because they define the basic opportunity: a receiver can reconstruct more bytes than the sender transmits when both sides agree on a dictionary and the reference semantics are unambiguous.

EndRE moved the idea toward endpoints and is therefore closer to ReduLink [4]. The difference is the transport era and the authentication burden. Modern QUIC endpoints cannot rely on transparent network rewriting because payload confidentiality, packet protection, and ossification resistance are design goals [6-10]. ReduLink treats the reference layer as an endpoint protocol above QUIC stream delivery rather than as a middlebox transformation below transport security.

LBFS and rsync-family tools demonstrate that file-oriented block reuse can be very effective over low-bandwidth links [5]. ReduLink is not a replacement for those tools. It asks a different systems question: whether a transport-compatible representation layer can carry authenticated FULL and REF messages over encrypted streams while preserving byte-exact reconstruction, semantic repair, and conservative accounting. The package therefore includes a fixed-block reuse approximation as an rsync-family baseline, but it does not implement the rsync protocol handshake or rolling checksum exchange.

Content-defined chunking and deduplicated storage literature explain both the promise and the risk of chunk identity [11-13]. CDC can survive insertions better than fixed blocks, but it introduces CPU cost and can expose chunk-presence side channels. Deduplication privacy work shows that cross-user or cross-tenant dictionaries must be treated as policy-sensitive state [16-17]. ReduLink therefore defaults to connection- or scope-bound dictionaries and reserves shared public dictionaries for explicit public-artifact cases.

The remaining gap is a deployable QUIC path. QUIC extension frames and transport parameters would be cleaner than application-stream messages, but they require stack changes and new interoperability testing. The current paper takes a narrower journal step: implement and measure the semantics over native encrypted QUIC streams first, then state exactly what remains open for extension-frame integration.

Table 1. Related-work positioning.

System class	Placement	Visibility	Dictionary scope	Strength	Gap for ReduLink
Classical RE [1-3]	Middlebox/path	Plaintext or observable bytes	Path or enterprise	Shows repeated traffic opportunity	Hard to deploy under encrypted QUIC
EndRE [4]	Endpoint	Endpoint plaintext	Endpoint/enterprise	Endpoint suppression model	Pre-QUIC framing and security assumptions

LBFS/rsync family [5]	File/application	File bytes	File pair or file cache	Strong delta-transfer baseline	Not a transport-compatible stream mapping
QUIC [6-10]	Transport	Encrypted payload	Connection state	Modern secure transport	No built-in redundancy references
CDC/dedup [11-13]	Storage/sync	Chunk identity	Dataset dependent	Shift-tolerant chunking	CPU cost and probing risks
Dedup privacy [16-17]	Storage/security	Chunk-presence signal	Tenant/user sensitive	Threat model guidance	Transport-scoped policy needed
ReduLink	Endpoint over QUIC streams	Endpoint plaintext before QUIC	Connection/scope/ public manifest	Authenticated stream-compatible references	Needs custom QUIC frames and netem validation

4. Protocol Model

ReduLink operates between cooperating endpoints. FULL frames carry original chunks and admit them to the receiver dictionary. REF frames carry compact authenticated references to chunks expected to be present. MISS messages identify references the receiver cannot validate or resolve. Repair FULL messages provide the missing bytes and allow byte-exact reconstruction to continue.

Table 2. ReduLink message semantics.

Message	Purpose	Receiver obligation
FULL	Carry literal bytes and dictionary metadata.	Validate tag, deliver bytes in order, admit scoped chunk.
REF	Refer to a previously admitted chunk.	Resolve exact chunk, validate binding, deliver only on success.
MISS	Report absent or invalid reference.	Do not deliver guessed bytes; request or await repair.
REPAIR FULL	Provide bytes for a missed reference.	Validate as FULL, fill semantic gap, resume stream.
ERROR	Abort invalid or policy-forbidden state.	Fail closed; preserve application byte integrity.

Invariant: a receiver may deliver reconstructed bytes only if the referenced dictionary entry exists, authentication succeeds, the epoch and scope match, stream id and reconstructed offset match, length matches, nonce state has not been replayed, and the per-frame hash agrees with the delivered bytes. This invariant is stronger than a plain cache lookup because a wrong reference is a data-integrity failure, not a cache miss.

5. Security and Privacy Model

The artifact validates HMAC-based bindings for tamper and replay rejection. Tests cover wrong secret, wrong epoch, wrong scope, wrong stream id, wrong offset, wrong length, modified payload, bad tag, and replayed nonce. Production deployment should derive ReduLink secrets from QUIC TLS exporter material rather than artifact-local random secrets [7]. The included HKDF schedule is an exporter-style stand-in that makes the binding rules executable and testable.

Table 3. Threats, controls, and remaining status.

Threat	Why it matters	Current control	Remaining work
Chosen-chunk probing	REF/MISS timing can reveal receiver content.	Connection-scoped dictionaries; no speculative private shared refs.	Constant-error policy and rate limits in production.
Cross-user leakage	Shared dedup can expose possession.	Shared dictionaries only for explicit public artifacts.	Tenant policy enforcement and audits.
Replay	Old valid REF could corrupt stream position.	Nonce, epoch, stream id, offset, and scope binding.	Transport-integrated replay windows.
Tamper	Modified payload or REF could alter bytes.	HMAC over semantic metadata and hashes.	QUIC TLS exporter integration.
Expansion abuse	Tiny refs could claim huge reconstruction.	Length binding and explicit dictionary entries.	Memory and ratio caps.
MISS storm	Bad references can force repair churn.	Fail-closed MISS/FULL repair semantics.	Rate limiting and sender accountability.
0-RTT replay	Early data may replay stale	Not enabled in artifact policy.	Transport-parameter negotiation.

	dictionary state.		
Migration confusion	Path changes may alter endpoint assumptions.	Not claimed.	Migration-safe dictionary epochs.

Privacy is not solved merely by encrypting the QUIC packets. A successful reference proves that the receiver has a chunk in the relevant dictionary. That is acceptable for connection-local state and for explicitly public version manifests, but it is dangerous for private cross-user dictionaries. ReduLink therefore treats dictionary scope as a security parameter rather than an optimization detail.

6. QUIC Stream-Mapping Implementation

The implementation uses aioquic to run a real QUIC handshake and encrypted bidirectional streams over localhost UDP [18]. ReduLink messages are carried as application-stream bytes using a compact binary format. This exercises QUIC packetization, ACK/loss machinery, stream flow control, TLS-protected streams, and application-level semantic repair without modifying aioquic internals.

The binary wire format is length-delimited and intentionally small: message type, stream context, reconstructed offset, chunk length, chunk identity, nonce, authentication tag, and optional literal bytes. The secure model separates encoding from verification so tests can validate both byte accounting and fail-closed behavior. The semantic-repair prototype intentionally withholds selected warm-dictionary entries at the receiver, emits MISS messages, and reconstructs after repair FULL frames.

6.1 Why Stream Mapping Is the Current Journal Step

A custom QUIC extension frame would be the cleaner long-term design because congestion control, ACK accounting, frame parsing, and negotiation could be handled at the transport layer. It is also a larger claim. A stream mapping is a deliberately modest step that reviewers can run with standard aioquic APIs. It demonstrates compatibility with encrypted QUIC streams and validates the representation semantics before requiring a modified QUIC stack.

Table 4. Implementation choices and claim strength.

Path	Implemented here?	What it proves	What it does not prove
Application STREAM mapping	Yes	Endpoint semantics over real encrypted QUIC streams.	Custom frame interoperability or packet-byte accounting.
Custom QUIC extension frames	No	Would reduce layering overhead and clarify transport negotiation.	Future work; needs stack modification.
UDP/TCP endpoint prototypes	Yes	Semantic repair and fail-closed behavior outside QUIC.	Native QUIC transport behavior.
Middlebox RE	No	Can optimize plaintext enterprise paths.	Incompatible with end-to-end encrypted QUIC payloads.

7. Evaluation Methodology

The evaluation combines four layers: workload byte-saving experiments, native QUIC stream experiments, security/failure tests, and local component costs. Baselines include raw transfer, gzip, zstd when available, fixed-block reuse as an rsync-family approximation, ReduLink fixed chunking, ReduLink CDC, authenticated ReduLink, and raw aioquic streams. All byte-saving claims use reconstructed bytes divided by encoded bytes at the stated measurement layer.

Table 5. Measurement definitions.

Metric	Meaning	Current limitation
input_bytes	Bytes the receiver must reconstruct.	Application payload, not file-system metadata.
warm_bytes	Prior receiver-side dictionary material.	Assumes endpoint-controlled availability.
aligned_changed_bytes	Byte positions changed under fixture alignment.	Diagnostic, not a universal diff metric.
wire_bytes	Modeled ReduLink or baseline encoded bytes.	Not always packet bytes.
stream_payload_bytes	Bytes placed into QUIC STREAM data.	Excludes QUIC/TLS/UDP/IP headers.
packet_bytes	Full encrypted packet bytes on the path.	Not captured in current artifact.
wall_ms	Local runner elapsed time.	Python artifact timing, not line-rate proof.
effective_multiplier	input_bytes divided by encoded bytes.	Layer must be stated.

Table 6. Baselines and comparators.

Comparator	Role	Interpretation
raw	No suppression or compression.	Lower bound for byte-saving comparison.
gzip/zstd	Single-object compression.	Often dominates logs or metadata.
fixed-block reuse	rsync-family approximation.	Exact block reuse; not full rsync protocol.
ReduLink fixed	Simple chunked references.	Best for aligned/page-like repeated data.
ReduLink CDC	Shift-tolerant reference search.	Better for some shifts but costly in Python.
secure HMAC ReduLink	Authenticated frame overhead.	Shows security cost and fail-closed checks.
aioquic raw/ReduLink	Native encrypted stream comparison.	Stream payload accounting, not packet capture.

7.1 Workload Construction and Validity

The package includes deterministic scripted journal fixtures because the review environment may be offline. These fixtures model page-aligned disk snapshots, OCI-like tar layers, package metadata, repository snapshots, structured logs, and independent compressed negatives. Their purpose is not to simulate the internet perfectly; it is to create repeatable workload families where chunk identity, alignment, and compression interaction can be inspected.

The v2.3 package adds independently curated public source-release pairs from Click, Redis, and nginx. These are real public artifacts with pinned URLs and archive hashes. They are still source-release snapshots, not production network traces, OCI registry layers, VM backups, or Git pack transfers. Their value is that they are external and mostly negative: they prevent the paper from overfitting its claim to scripted fixtures.

8. Workload Results

Table 7. Deterministic journal workload suite, warm-update mode.

Workload	Chunker	Input	Warm	ReduLink bytes	ReduLink	Fixed-block	Recon.
disk-snapshot	fixed	1,048,576	1,048,576	36,808	28.488x	31.973x	OK
oci-layer	fixed	296,960	296,960	26,864	11.054x	10.682x	OK
package-metadata	fixed	526,287	526,191	529,383	0.994x	9.880x	OK
repository-snapshot	cdc	501,760	501,760	99,144	5.061x	24.596x	OK
structured-logs	fixed	4,267,415	4,267,340	4,292,423	0.994x	14.887x	OK
independent-compressed-negative	fixed	262,242	262,242	263,802	0.994x	1.000x	OK

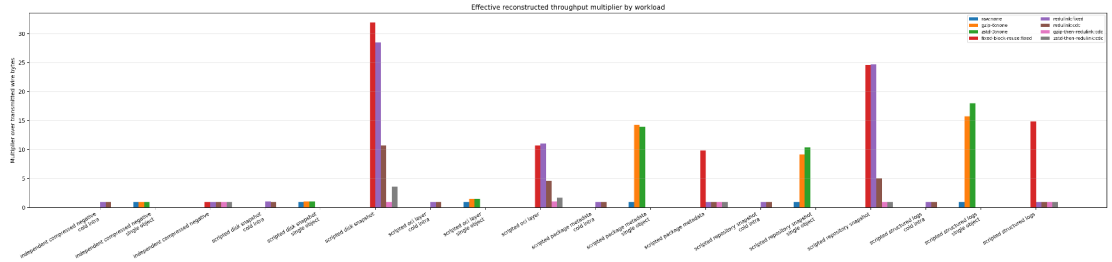

Figure 1. Journal workload effective multipliers in warm-update mode.

The deterministic workload evidence is mixed, as expected. Disk snapshots and OCI-like layers benefit when fixed boundaries preserve byte identity. Repository-like data shows a modest CDC gain. Package metadata and structured logs are weak because small edits shift or disperse byte identity, while single-object compression often captures the repetition more effectively than references. The random and compressed negative controls correctly show no meaningful gain.

8.1 External Public Source-Release Results

Table 8. Independently curated public source-release pairs.

Public pair	New bytes	ReduLink fixed	Secure	Fixed-block	Interpretation
click-8.1.7-to-8.1.8	953,008	0.994x	0.986x	0.992x	No gain; overhead exceeds reference savings.
redis-7.2.4-to-7.2.5	15,533,278	0.998x	0.990x	0.996x	No gain; overhead exceeds reference savings.
nginx-1.25.3-to-1.25.4	7,377,697	0.997x	0.989x	0.995x	No gain; overhead exceeds reference savings.

The external public corpus is deliberately sobering. Click 8.1.7 to 8.1.8, Redis 7.2.4 to 7.2.5, and nginx 1.25.3 to 1.25.4 all reconstruct correctly, but fixed 4 KiB ReduLink yields multipliers below 1.0. The result is useful because it falsifies an overbroad claim: ordinary related source releases do not automatically contain enough boundary-stable byte identity to justify reference substitution.

8.2 Negative Results and Why ReduLink Loses

ReduLink loses when overhead is larger than the saved bytes, when chunk boundaries shift, when compression has already removed the repeated structure, or when semantically related content is not byte-identical. Package metadata and structured logs are examples where field-level similarity does not guarantee chunk-level identity. Source-release trees add another negative case: file additions, timestamps, generated files, and small edits can move enough bytes that a simple fixed-boundary chunker has little to reference.

These negative cases strengthen the paper because they define the workload gate. ReduLink is most credible for VM/page-like snapshots, aligned container layers, repeated binary objects, and explicitly versioned public artifacts with stable byte regions. It is least credible as a generic text-diff or compression replacement.

9. Native QUIC Flow Comparison

Table 9. Raw QUIC versus ReduLink binary stream mapping.

Method	Loss every	Input	Stream payload	Multiplier	MISS	Repair	Recon.
raw-quic-stream	0	98,304	98,304	1.000x	0	0	OK
redulink-binary-quic-stream	0	98,304	30,816	3.190x	13	13	OK
raw-quic-stream	9	98,304	98,304	1.000x	0	0	OK
redulink-binary-quic-stream	9	98,304	30,816	3.190x	13	13	OK

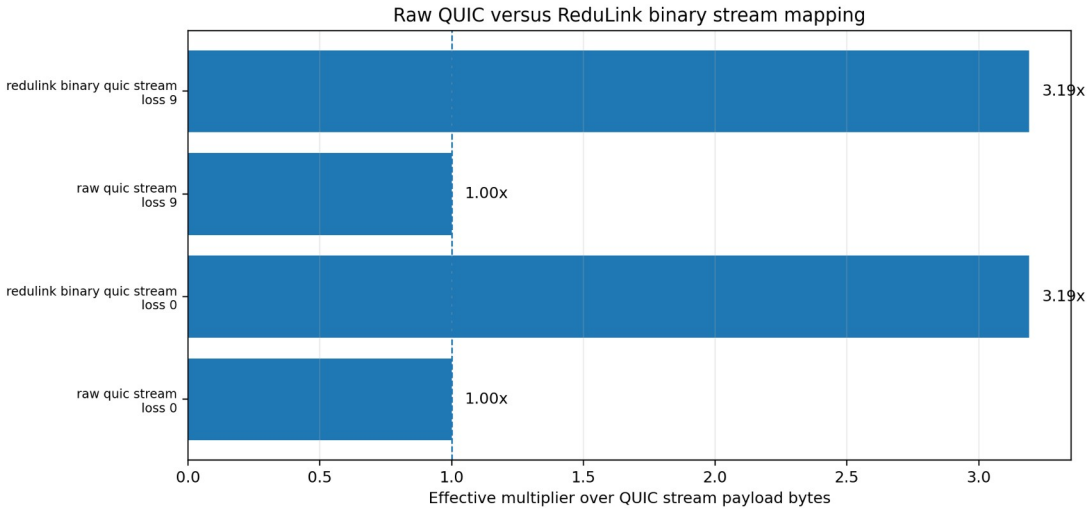

Figure 2. Raw QUIC versus ReduLink binary stream mapping.

The native aioquic comparison transfers the same 98,304-byte update as raw QUIC and as ReduLink binary stream mapping. Raw QUIC uses 98,304 stream-payload bytes, while the ReduLink stream mapping uses about 30 KiB after semantic repairs, yielding about a 3.19x stream-payload multiplier in the comparison CSV. Under deterministic proxy loss, both paths reconstruct byte-exactly, and ReduLink repairs semantic misses with FULL messages rather than delivering incorrect bytes.

This is stream-payload evidence. It does not count QUIC packet headers, UDP/IP headers, TLS record expansion, or packet coalescing effects. The correct interpretation is that the application placed fewer bytes into QUIC STREAM frames, not that the full path-byte cost has been measured with packet capture.

Table 10. Accounting layers.

Layer	Definition	Measured here?
Original application bytes	Bytes in the update to reconstruct.	Yes
Reconstructed bytes	Bytes delivered after FULL/REF validation.	Yes
ReduLink encoded bytes	Modeled FULL/REF message bytes.	Yes
QUIC stream payload bytes	Bytes submitted as stream data.	Yes
QUIC packet bytes	Encrypted QUIC datagrams on UDP.	No
UDP/IP/path bytes	Full on-path bytes including lower headers.	No

The paper therefore uses M_stream = input_bytes / stream_payload_bytes for aioquic comparisons. A later packet-capture study should report M_packet = input_bytes / packet_bytes and include competing flows under tc/netem or Mininet.

10. Component-Cost Results

Table 11. Local Python component-cost measurements.

Component	Input	Wall ms	MiB/s local	Scope
fixed_chunking	2,097,152	0.321	6237.000	local Python artifact timing; not production line-rate evidence
cdc_chunking	2,097,152	267.099	7.488	local Python artifact timing; not production line-rate evidence
model_fixed_encode_decode	1,573,964	2.207	680.015	local Python artifact timing; not production line-rate evidence
model_cdc_encode_decode	1,573,964	443.970	3.381	local Python artifact timing; not production line-rate evidence
secure_hmac_encode_roun	1,573,964	7.811	192.181	local Python artifact timing;

dtrip				not production line-rate evidence
binary_wire_encode	1,573,964	0.320	4696.898	local Python artifact timing; not production line-rate evidence
binary_wire_decode	1,573,964	0.612	2451.193	local Python artifact timing; not production line-rate evidence

The component-cost table is conservative. Fixed chunking and binary encode/decode are fast in the Python artifact, while CDC is much slower. These timings are local Python measurements and do not establish line-rate feasibility. A production implementation would need native chunking, bounded dictionary lookups, backpressure integration, and careful memory accounting.

11. Fairness and Transport Semantics

ReduLink separates encoded stream bytes from reconstructed application bytes. Congestion control should see only the bytes placed into QUIC, while the application observes reconstructed bytes after validation. This distinction is essential: a protocol that receives congestion credit for reconstructed bytes would be unfair. The current artifact supports the accounting rule but does not complete the congestion-control study.

Table 12. Fairness evidence ladder.

Evidence	File	Supports	Does not support
Wire-byte accounting	results/ wire_fairness_accounting.csv	ReduLink wire share 0.083 uses encoded bytes.	Real QUIC congestion dynamics.
Concurrent localhost QUIC smoke	results/quic_competing_flows.csv	ReduLink stream payload 98,304 reconstructed with lower encoded stream bytes.	Controlled bottleneck or fairness under queueing.
Portable bottleneck emulation	results/ quic_bottleneck_emulation.csv	Fluid fair-share model over measured stream payload bytes.	Kernel tc/netem, ACK pacing, or loss coupling.
Needed next	not included	Packet capture plus multi-flow netem/Mininet.	Current package cannot claim this.

The bottleneck-emulation analysis applies a shared fluid-service model to measured raw QUIC and ReduLink-over-QUIC stream-payload bytes at several rates and RTT assumptions. This portable analysis is useful for checking that lower encoded bytes would reduce service demand, but it is not a substitute for live competing-flow experiments with kernel queues, packet capture, and congestion-control instrumentation.

12. Reproducibility Package

The package is designed so reviewers can reproduce the evidence without private data. It includes source code, tests, deterministic corpus generators, public-corpus fetchers, CSV outputs, figures, citation checks, and an inventory. The external public source release suite records codeload URLs, archive sizes, archive SHA-256 values, extracted paths, and per-pair reconstruction results.

Table 13. Reproducibility commands.

Purpose	Command
Unit and security tests	python3 -m unittest discover -s tests
Citation audit	python3 scripts/check_manuscript_citations.py
Generated artifact check	python3 benchmarks/check_generated_artifacts.py
Journal fixtures	bash benchmarks/run_journal_workload_suite.sh
External public corpora	python3 benchmarks/fetch_external_public_corpora.py
External public suite	python3 benchmarks/run_real_workload_manifest.py --manifest benchmarks/external_public_manifest.csv --output results/external_public_suite.csv
QUIC flow comparison	python3 benchmarks/run_quic_flow_comparison.py

13. Limitations and Journal Roadmap

- The implementation is a native QUIC stream mapping, not custom QUIC extension-frame parsing.
- The new external corpus consists of source-release snapshots; it is not a substitute for OCI registry layers, VM backups, Git pack traces, package repository metadata, or production logs.
- The fixed-block baseline is an rsync-family approximation, not the full rsync protocol or zsync.
- The Python CDC implementation is not line-rate and should not be interpreted as production performance.
- Fairness is supported by accounting, raw-flow comparison, concurrent localhost smoke tests, and bottleneck emulation, but not by a full tc/netem or Mininet multi-flow congestion-control study.
- The artifact HKDF schedule should be replaced by direct QUIC TLS exporter integration in a production implementation.
- 0-RTT dictionary policy, connection migration, transport-parameter negotiation, packet capture, and memory-exhaustion controls remain open implementation work.

The strongest next experiment is therefore not another scripted fixture. It is a live network-emulator study with raw QUIC, ReduLink stream mapping, and at least one competing flow under controlled bottlenecks, combined with packet-byte accounting. The strongest workload addition is a public, independently curated corpus for one target class such as OCI layers or VM snapshots where ReduLink is expected to help.

14. Conclusion

ReduLink demonstrates that authenticated reference substitution can increase effective reconstructed throughput for selected redundant warm-state workloads over encrypted endpoint transports. The current artifact provides native aioquic stream mapping, compact binary messages, authenticated fail-closed semantics, public and deterministic workload evidence, negative controls, local component measurements, and reproducibility checks. The strongest results occur when byte-stable chunks survive between warm and update states; the weakest results occur when related content lacks stable byte identity.

The appropriate journal claim is therefore precise rather than expansive. ReduLink is a credible endpoint-layer mechanism and evaluation artifact for redundancy-suppressed encrypted transfers. It is not yet a production QUIC extension, a universal accelerator, or a replacement for rsync, compression, or congestion-control experiments. That boundary makes the contribution more defensible and gives a clear path for the next systems revision.

References

[1] N. T. Spring and D. Wetherall, 'A protocol-independent technique for eliminating redundant network traffic,' ACM SIGCOMM, 2000.

[2] A. Anand, V. Sekar, and A. Akella, 'SmartRE: An architecture for coordinated network-wide redundancy elimination,' ACM SIGCOMM, 2009.

[3] A. Anand et al., 'Redundancy in network traffic: Findings and implications,' ACM SIGMETRICS, 2009.

[4] B. Aggarwal et al., 'EndRE: An end-system redundancy elimination service for enterprises,' USENIX NSDI, 2010.

[5] A. Muthitacharoen, B. Chen, and D. Mazieres, 'A low-bandwidth network file system,' ACM SOSP, 2001.

[6] J. Iyengar and M. Thomson, 'QUIC: A UDP-based multiplexed and secure transport,' RFC 9000, IETF, 2021.

[7] M. Thomson and S. Turner, 'Using TLS to secure QUIC,' RFC 9001, IETF, 2021.

[8] J. Iyengar and I. Swett, 'QUIC loss detection and congestion control,' RFC 9002, IETF, 2021.

[9] M. Kuehlewind and B. Trammell, 'Applicability of the QUIC transport protocol,' RFC 9308, IETF, 2022.

[10] B. Trammell et al., 'Manageability of the QUIC transport protocol,' RFC 9312, IETF, 2022.

[11] Y. Hu et al., 'The design of fast content-defined chunking for data deduplication,' IEEE Transactions on Parallel and Distributed Systems, 2020.

[12] M. Gregoriadis, L. Balduf, B. Scheuermann, and J. Pouwelse, 'A thorough investigation of content-defined chunking algorithms for data deduplication,' arXiv, 2024.

[13] B. Alexeev, C. Percival, and Y. X. Zhang, 'Chunking attacks on file backup services using content-defined chunking,' arXiv, 2025.

[14] IEEE 802.3 Ethernet Working Group, 'IEEE 802.3 Ethernet Working Group active projects,' accessed June 2026.

[15] M. Nguyen, 'ReduLink repository and reproducibility package,' GitHub repository, 2026.

[16] D. Harnik, B. Pinkas, and A. Shulman-Peleg, 'Side channels in cloud services: Deduplication in cloud storage,' IEEE Security & Privacy, 2010.

[17] M. Bellare, S. Keelveedhi, and T. Ristenpart, 'DupLESS: Server-aided encryption for deduplicated storage,' USENIX Security, 2013.

[18] J. Shi and contributors, 'aioquic: QUIC and HTTP/3 implementation in Python,' software project, 2026.

Appendix A: Package Contents

The submission package contains source code under src/, runnable prototypes under prototypes/, benchmark drivers under benchmarks/, generated evidence under results/, figures under figures/, and submission-ready manuscript files under paper/submission/.

Appendix B: External Public Corpus Manifest

Label	Old archive bytes	New archive bytes	Old SHA-256 prefix	New SHA-256 prefix
click-8.1.7-to-8.1.8	341,625	342,860	89251974dba8	33b16b4899d3
redis-7.2.4-to-7.2.5	3,424,072	3,427,473	0a62b9ae89b4	98a8502a2e90
nginx-1.25.3-to-1.25.4	1,228,791	1,230,692	e43252bb993e	a4d685701fd1